Name:Krishna Dalvi

D15C/22

Experiment – 10

---

1. Aim

To explore and implement a Convolutional Autoencoder for removing noise from images using the MNIST dataset.

---

2. Dataset Source

Source: Kaggle – Digit Recognizer (MNIST Dataset)
Link: https://www.kaggle.com/c/digit-recognizer

---

3. Dataset Description

The MNIST dataset consists of **28 × 28 grayscale images** of handwritten digits ranging from **0 to 9**. In this experiment, it is used for an **unsupervised learning** task where the labels are discarded. The model learns to reconstruct clean images from artificially corrupted ones.

Variable Name	Data Type	Measuring Unit / Format	Description
Input ((X_{noisy}))	Numerical	Pixel Intensity (0–1)	The corrupted handwritten digit infused with Gaussian noise.
Target ((X_{clean}))	Numerical	Pixel Intensity (0–1)	The original, uncorrupted handwritten digit used as ground truth.

---

4. Mathematical Formulation of the Algorithm

An **Autoencoder** is an unsupervised neural network that learns a compressed representation of data and reconstructs it with minimal loss of information. It consists of two main components: the **Encoder** and the **Decoder**.

A. Encoder

The encoder compresses the high-dimensional input image into a lower-dimensional latent representation.

$$
h = f(W_e x + b_e)
$$

Where:

- (x) = Input image
- (h) = Latent representation
- (W_e) = Encoder weights
- (b_e) = Encoder bias
- (f) = Activation function (ReLU)

B. Decoder

The decoder reconstructs the image from the latent representation.

$$
r = g(W_d h + b_d)
$$

Where:

- (r) = Reconstructed image
- (W_d) = Decoder weights
- (b_d) = Decoder bias
- (g) = Activation function (Sigmoid)

C. Loss Function

The objective is to minimize the reconstruction error between the original clean image and the reconstructed image. Since pixel values lie between 0 and 1, **Binary Cross-Entropy (BCE)** is used:

$$
L = -\frac{1}{N} \sum_{i=1}^{N} \left[x_i \log(r_i) + (1 - x_i)\log(1 - r_i) \right]
$$

Where:

- x_i = Original pixel value
- r_i = Reconstructed pixel value
- N = Total number of pixels

---

5. Methodology / Workflow

Step 1: Data Ingestion

- Load the MNIST dataset using TensorFlow/Keras.
- Discard categorical labels.

Step 2: Preprocessing

- **Normalization:** Scale pixel values by dividing by 255.0.
- **Reshaping:** Convert images to shape **(28, 28, 1)** for convolutional processing.

Step 3: Noise Injection

- Add Gaussian noise to simulate corrupted images:

$$
X_{noisy} = X_{clean} + \alpha \cdot \mathcal{N}(0,1)
$$

Where:

- $\alpha = 0.5$ is the noise factor.
- Clip values to maintain valid pixel ranges:

```
np.clip(image, 0., 1.)
```

Step 4: Model Architecture Design

Encoder

- Conv2D (32 filters, 3×3, ReLU, padding='same')

- MaxPooling2D (2×2)

Decoder

- Conv2D (32 filters, 3×3, ReLU, padding='same')
- UpSampling2D (2×2)

Output Layer

- Conv2D (1 filter, Sigmoid activation)

Step 5: Compilation and Training

- **Optimizer:** Adam
- **Loss Function:** Binary Cross-Entropy
- **Epochs:** 5
- **Batch Size:** 128

```
model.compile(optimizer='adam', loss='binary_crossentropy')
model.fit(x_train_noisy, x_train,
        epochs=5,
        batch_size=128,
        validation_data=(x_test_noisy, x_test))
```

Step 6: Evaluation

- Predict denoised images from noisy inputs.
- Visualize results using Matplotlib.

```
decoded = model.predict(x_test_noisy)
plt.imshow(decoded[0].reshape(28, 28), cmap='gray')
plt.show()
```

---

6. Performance Analysis

- **Denoising Capability:**
  The autoencoder effectively removes Gaussian noise while preserving the structure of handwritten digits.

- **Unsupervised Learning:**
  The model learns meaningful features without requiring labeled data.
- **Feature Extraction:**
  Continuous strokes are identified as signal, while random scattered pixels are treated as noise.

---

7. Hyperparameter Tuning (Architecture Selection)

Hyperparameter	Value	Justification
Noise Factor	0.5	Provides sufficient corruption for robust learning.
Filters	32	Captures edges and curves effectively.
Kernel Size	3×3	Ideal for extracting spatial features.
Activation	ReLU/Sigmoid	Ensures non-linearity and normalized output.
Padding	Same	Preserves spatial dimensions.
Optimizer	Adam	Enables efficient convergence.
Epochs	5	Suitable for quick experimentation.
Batch Size	128	Balances speed and memory efficiency.

---

8. Algorithm Limitations

- **Domain Specificity:**
  The model performs well only on data similar to MNIST.
- **Blurriness in Reconstruction:**
  BCE and MSE losses tend to produce slightly blurred outputs.
- **Information Loss:**
  A very small latent space may discard essential image details.
- **Limited Generalization:**
  Performance decreases on complex real-world images.

---

9. Code & Output

```
# ==========================================================
```

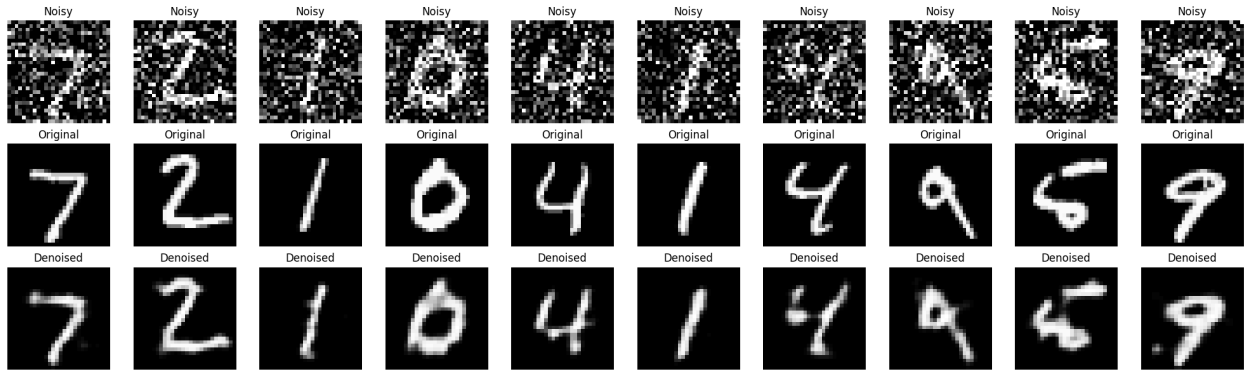
```
# Experiment 10: Autoencoders for Image Denoising
# ============================================================
# Step 1: Import Required Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
# Set random seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
# ============================================================
# Step 2: Load and Preprocess the Dataset
# ============================================================
(x_train, _), (x_test, _) = mnist.load_data()
# Normalize pixel values between 0 and 1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# Reshape images to include channel dimension
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
print("Training Data Shape:", x_train.shape)
print("Testing Data Shape:", x_test.shape)
```

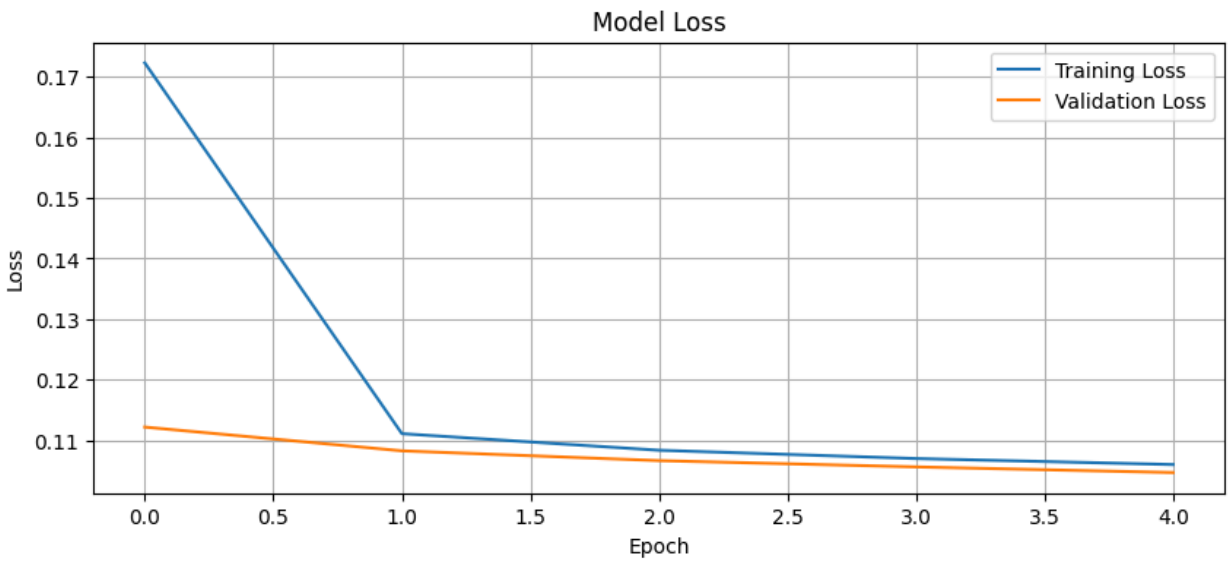
```
# ============================================================
# Step 3: Add Gaussian Noise
# ============================================================
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(
loc=0.0, scale=1.0, size=x_train.shape
)
x_test_noisy = x_test + noise_factor * np.random.normal(
loc=0.0, scale=1.0, size=x_test.shape
)
# Clip values to maintain pixel range [0, 1]
x_train_noisy = np.clip(x_train_noisy, 0.0, 1.0)
x_test_noisy = np.clip(x_test_noisy, 0.0, 1.0)
# ============================================================
# Step 4: Build the Convolutional Autoencoder
# ============================================================
input_img = Input(shape=(28, 28, 1))
# ---------------- Encoder ----------------
x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
# ---------------- Decoder ----------------
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = UpSampling2D((2, 2))(x)
# Output Layer
```

```
output = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Define the model

autoencoder = Model(inputs=input_img, outputs=output)

# Display model summary

autoencoder.summary()

# ============================================================

# Step 5: Compile the Model

# ============================================================

autoencoder.compile(

optimizer='adam',

loss='binary_crossentropy',

metrics=['accuracy']

)

# ============================================================

# Step 6: Train the Model

# ============================================================

history = autoencoder.fit(

x_train_noisy,

x_train,

epochs=5,

batch_size=128,

shuffle=True,

validation_data=(x_test_noisy, x_test)

)
```

```
# ============================================================
# Step 7: Test the Model
# ============================================================
decoded_images = autoencoder.predict(x_test_noisy)
# ============================================================
# Step 8: Visualize the Results
# ============================================================
n = 10 # Number of digits to display
plt.figure(figsize=(20, 6))
for i in range(n):
# Display Noisy Images
ax = plt.subplot(3, n, i + 1)
plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
plt.title("Noisy")
plt.axis("off")
# Display Original Images
ax = plt.subplot(3, n, i + 1 + n)
plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
plt.title("Original")
plt.axis("off")
# Display Denoised Images
ax = plt.subplot(3, n, i + 1 + 2 * n)
plt.imshow(decoded_images[i].reshape(28, 28), cmap='gray')
plt.title("Denoised")
```

```
plt.axis("off")

plt.tight_layout()

plt.show()

# ============================================================

# Step 9: Plot Training History

# ============================================================

plt.figure(figsize=(10, 4))

# Plot Loss

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Model Loss')

plt.xlabel('Epoch')

plt.ylabel('Loss')

plt.legend()

plt.grid(True)

plt.show()

# ============================================================

# Step 10: Save the Model

# ============================================================

autoencoder.save("autoencoder_denoising_mnist.h5")

print("Model saved successfully!")
```





---

10. Conclusion

A **Convolutional Autoencoder** was successfully implemented to denoise MNIST digit images. The model demonstrated effective unsupervised feature extraction by reconstructing clean images from noisy inputs. This experiment highlights the power of deep learning in image restoration and dimensionality reduction, with applications in medical imaging, surveillance, and computer vision.